MODULE 1

BLUETOOTH CONNECTIVITY

OBJECTIVES

1) Understand Bluetooth technology.
2) Understand inter-device communication using Bluetooth.
3) Implement Bluetooth in an IoT system.

REQUIREMENTS

1) Arduino IDE/PlatformIO,
2) ESP32,
3) Sensor (optional),
4) Actuator (optional).

Bluetooth connectivity can be used for direct communication between two devices. Peer-to-peer (P2P) is a communication pattern in which devices interact directly without a central application server. However, Bluetooth role terminology depends on the technology: Bluetooth Classic may use initiator and acceptor roles for particular connections, whereas Bluetooth Low Energy uses Central and Peripheral roles in the Generic Access Profile (GAP).

1.1 Bluetooth

Bluetooth is a wireless communication technology used to connect one communication device to another without cables. It is commonly used in mobile devices, laptops, computers, and other equipment.

Figure 1.1: Bluetooth logo

Bluetooth technology was developed in the 1990s by a team of Ericsson engineers, including Jaap Haartsen. The name Bluetooth was taken from the epithet of King Harald “Blåtand” Gormsson, who is associated with the unification of Denmark and Norway. The name was adopted as a symbol of uniting different devices through a single wireless communication technology.



Figure 1.2: Jacobus Cornelis Haartsen

Bluetooth is used for short-range data exchange and to connect devices such as headsets, speakers, keyboards, sensors, and computers. Some Bluetooth profiles also support networking functions, including tethering.

Bluetooth operates in the 2.4 GHz ISM band. Bluetooth Classic uses frequency hopping, whereas Bluetooth Low Energy uses different channels and radio-access mechanisms. Range, speed, and power consumption are affected by the version, PHY mode, transmit power, antenna, environmental conditions, and device implementation.

1.2 Bluetooth Versions

1.2.1 Bluetooth 1.0

In 1999, the first Bluetooth release encountered numerous problems, and manufacturers had difficulty producing devices that could interoperate correctly. Version 1.0 and its amended release, 1.0B, therefore had limited practical success.

Bluetooth 1.1 was released in 2001 with improvements over its predecessor. Bluetooth 1.1 was also reflected in the IEEE 802.15.1-2002 standard.

Bluetooth 1.2 was introduced in 2003 with improvements to discovery, Adaptive Frequency Hopping (AFH), and connection quality. Bluetooth devices are designed to retain interoperability with features from earlier versions, although a successful connection still depends on the profiles and features supported by both devices.

1.2.2 Bluetooth 2.0

Bluetooth 2.0 was introduced in 2004 with Enhanced Data Rate (EDR) as an optional feature for increasing data rates and transmission-time efficiency. EDR did not remove Basic Rate support, so devices could continue communicating through mutually supported features.

Three years later, Bluetooth 2.1 was released. It introduced Secure Simple Pairing (SSP) to make pairing faster and more secure. It also introduced Extended Inquiry Response (EIR), which helps users obtain information about a device before connecting and supports more efficient discovery behavior.

1.2.3 Bluetooth 3.0

In 2009, Bluetooth 3.0 + High Speed (HS) introduced the Alternate MAC/PHY (AMP). The Bluetooth channel is used for discovery, connection, and management, while high-speed data transfer can be moved to an IEEE 802.11 radio on devices that support the HS feature. The promoted theoretical rate was approximately 24 Mbps, although the feature was optional.

1.2.4 Bluetooth 4.0 (Bluetooth Low Energy)

In 2010, Bluetooth Core Specification 4.0 introduced Bluetooth Low Energy (BLE) as a low-power radio system alongside Bluetooth Classic. BLE was designed to allow devices to operate with low energy consumption, including devices powered by small batteries. BLE 4.x uses the LE 1M PHY with a symbol rate of 1 Msym/s; application throughput and actual range depend on the configuration and environmental conditions.

Bluetooth 4.1, adopted in 2013, improved coexistence with cellular networks, device-role flexibility, and connection management. These improvements do not mean that a Bluetooth connection is automatically always on or off; power behavior is still determined by the device and application implementation.

Bluetooth 4.2, adopted in 2014, added improvements to BLE data-packet capacity, privacy, connection security, and support for the Internet Protocol Support Profile. Throughput improvements depend on both devices supporting the Data Length Extension feature and should not be equated with internet download speed.

1.2.5 Bluetooth 5.0

Bluetooth Core Specification 5.0 was adopted in 2016 and introduced important enhancements for Bluetooth Low Energy, including the LE 2M PHY, LE Coded PHY, and Extended Advertising. These features support a range of IoT requirements, but security and interference resilience still depend on the device configuration and implementation.

The three principal features introduced were:

- Up to twice the data rate through the LE 2M PHY
- Up to four times the range through the LE Coded PHY under particular conditions
- Up to eight times the advertising-data capacity compared with legacy advertising

Bluetooth 5.0 retains support for the LE 1M PHY and adds the LE 2M PHY for higher data rates. The LE Coded PHY can extend range by adding redundancy, but this reduces the effective data rate. Devices can use these features only when both sides support and enable them.

1.2.6 Bluetooth 6.0

The newest major version of Bluetooth is released on 2024, focused on precision ranging, lower latency, and efficiency improvement on device discovery. The latest feature from this release is Channel Sounding, where the bluetooth protocol will use precision ranging based on phase-change and time difference between arrival and departure. This feature enable precision ranging within centimeter accuracy. Although Bluetooth 6.0 doesn't provide improvement in data transfer rate, it shows us the improvement in power efficiency and better security compared to the previous version.

1.3 Bluetooth Communication Flow

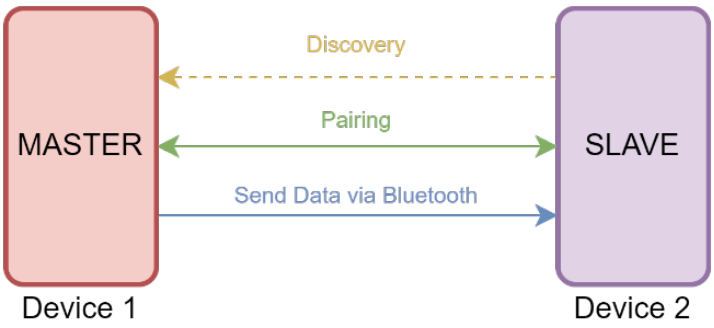


Figure 1.3: Bluetooth communication flow

The Bluetooth operating sequence is shown in the figure above and is explained as follows:

a. **Discovery**

A device performs discovery to find other devices within range. Bluetooth Classic may use the inquiry procedure, whereas Bluetooth Low Energy scans advertising packets.

b. **Pairing**

Pairing may be performed after a device has been discovered when the application requires key establishment and a trusted relationship. Not all Bluetooth connections require pairing; security requirements are determined by the profile, service, and device configuration.

c. **Connection**

The devices may then establish a connection to exchange data. Connection roles do not determine the direction of data transfer: both devices may transmit and receive. Bluetooth Low Energy uses the terms Central and Peripheral, whereas Bluetooth Classic role terminology depends on the procedure and profile used.

1.4 Bluetooth Protocols

The Bluetooth Special Interest Group (SIG) develops the Bluetooth specifications, which define the protocols used by Bluetooth technology. Fundamental Bluetooth Classic components include the Bluetooth Radio, Baseband, and Link Manager Protocol (LMP), which form part of the core protocol stack. Higher-level adopted protocols can be integrated into the Bluetooth protocol architecture and may originate from

other organizations. The radio, baseband, and LMP broadly correspond to physical- and data-link-layer functions in the OSI model.

The Bluetooth protocol stack can be grouped into four layers according to their functions. The protocols included in these layers are shown in the following table:

Table 1.1: Bluetooth protocol layers and protocol stack

Protocol Layer	**Protocols in the Stack**
Bluetooth Core Protocols	Baseband, LMP, L2CAP, SDP
Cable Replacement Protocols	RFCOMM
Telephony Control Protocols	TCS Binary, AT-commands
Adopted Protocols	PPP, UDP/TCP/IP, OBEX, WAP, vCard, vCal, IrMC, WAE

The Host Controller Interface (HCI) additionally provides commands and events between the host and controller, supporting baseband control, connection management, and access to controller functions.

a. **Baseband**

This layer enables RF links among Bluetooth units that form a piconet. The Bluetooth Classic radio system uses frequency-hopping spread spectrum, transmitting data as packets in defined time slots and on defined frequencies. It performs inquiry and paging procedures to synchronize frequency-hopping transmission and the clocks of different Bluetooth devices.

b. **Link Manager Protocol (LMP)**

The Link Manager Protocol manages link establishment and operation in Bluetooth Classic. Its functions include authentication, encryption setup, link-parameter negotiation, and control of baseband features.

c. **Logical Link Control and Adaptation Protocol (L2CAP)**

L2CAP packets carry payloads to higher protocol layers.

d. **Service Discovery Protocol (SDP)**

This protocol is used to discover services provided by a Bluetooth Classic device and the attributes required to access those services.

e. **Cable Replacement Protocol (RFCOMM)**

RFCOMM provides serial-port emulation.

f. **Telephony Control Protocol**

Telephony Control Specification Binary (TCS BIN) and AT commands are used in particular telephony profiles to control calls and modem functions.

g. **Adopted Protocols**

Bluetooth also supports adopted protocols such as PPP, TCP/UDP/IP, OBEX, and WAP to improve interoperability.

h. **Radio Frequency (RF)**

Radio Frequency is the lowest layer of the Bluetooth specification and provides wireless communication in the 2.4 GHz Industrial, Scientific, and Medical (ISM) band. Bluetooth Classic uses 79 RF channels, each 1 MHz wide, with frequency hopping, whereas Bluetooth Low Energy uses 40 RF channels, each 2 MHz wide.

1.5 Implementing Peer-to-Peer Bluetooth

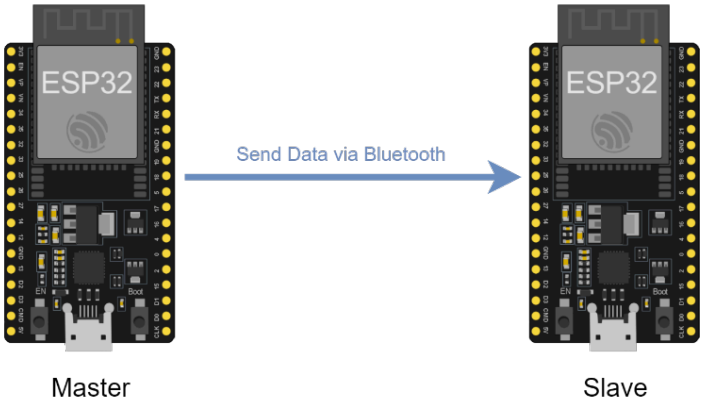


Figure 1.4: P2P visualization

In this peer-to-peer Bluetooth implementation, the master sends data to the slave using ESP32 microcontrollers. If you want to use other microcontroller, make sure the device support the required Bluetooth version. When the device only support Bluetooth with version 4.0 and above without dual-mode support, then the device can only use BLE implementation and vice-versa. If the device has dual-mode support (Bluetooth Classic and BLE), the device can use both implementation of Bluetooth Classic and BLE.

1.5.1 Using Bluetooth Serial

Bluetooth Serial configuration is relatively straightforward compared with BLE because it uses **Master** and **Slave** roles similar to other serial communication protocols. The following examples show master and slave configurations on the ESP32.

Information
The Bluetooth Serial library used here is the ESP32 built-in library:
https://github.com/espressif/arduino-esp32/tree/master/libraries/BluetoothSerial

- **Master**

  The Master searches for a connection to the Slave and sends data to it.

Master/Host Source-Code Initialization

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String masterName = "ESP32-Master"; // ESP Master name
String slaveName = "ESP32-Slave";
bool connected = false;
int value = 0;
```

To initialize ESP32 P2P communication, the `BluetoothSerial.h` library is used to access Bluetooth functions. The required variables are then declared: `SerialBT` provides access to BluetoothSerial, `String masterName` defines the master's visible name, `String slaveName` identifies the target device `ESP32-Slave`, `bool connected` stores the connection status, and `int value` stores the value to be sent to the Slave.

Master Source-Code Setup

```
void setup() {
  Serial.begin(115200);

```

```
    SerialBT.begin(masterName, true); // Start Bluetooth
    Serial.println("The device started in master mode, make sure remote BT device is on!");

    connected = SerialBT.connect(slaveName); // Check the Bluetooth connection
    if(connected) {
      Serial.println("Connected Successfully!");
    } else {
      while(!SerialBT.connected(10 * 1000)) { // Wait for 10 seconds
        Serial.println("Failed to connect!");
        Serial.println("Make sure remote device is available and in range, then restart");
      }
    }
    SerialBT.connect();
}
```

In `setup()`, `SerialBT.begin(masterName, true)` starts Bluetooth in master mode and assigns the name visible to other devices. `connected = SerialBT.connect(slaveName)` attempts to connect to the target Bluetooth device and records whether the connection succeeds. The subsequent `SerialBT.connect();` call can be used to request reconnection.

Master Source-Code Loop

```
void loop() {
  if(value <= 10){
    SerialBT.println(value); // Send data through Bluetooth
    Serial.println(value);
    value++;

    delay(1000);
  }
  else {
    value = value - 10;
  }
}
```

In the loop, the program sends the value of the `value` variable to the `Slave` until it reaches 10. The statement `SerialBT.println(value)` sends the current value of the `value` variable to the `Slave`.

- **Slave**

  The Slave receives communication from the Master.

Slave Source-Code Initialization

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // ESP Slave name
int value = 0;
String slaveValueStr;
int slaveValueInt;
```

The Slave initialization also uses the `BluetoothSerial.h` library. A `BluetoothSerial` object is assigned to `SerialBT`, and the required variables are declared: `int value`, `String slaveValueStr` for received string data, and `int slaveValueInt` for storing the integer value converted from that string. Finally, `String slaveName = "ESP32-Slave"` defines the device name.

Slave Source-Code Setup

```
void setup() {
  Serial.begin(115200);

```

```
    SerialBT.begin(slaveName); // Start Bluetooth as a receiving device
    Serial.println("The device started, now you can pair it with bluetooth!");
}
```

During setup, `SerialBT.begin(slaveName)` starts the Bluetooth service and sets the name that will be visible to other devices from the value of `slaveName`.

Slave Source-Code Loop

```
void loop() {
  if(SerialBT.available()){
    slaveValueStr = SerialBT.readStringUntil('\n'); // Read data from the Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data from Master: ");
    Serial.println(slaveValueInt);
  }
}
```

In the loop, `if(SerialBT.available())` checks whether data from the master is available over Bluetooth. The `slaveValueStr` variable reads the incoming string up to the newline character with `SerialBT.readStringUntil('
')`. The received `String` is then converted to an `int` with `slaveValueStr.toInt()` and stored in `slaveValueInt`, producing the following Serial Monitor output.

```
Data Dari Master : 1
Data Dari Master : 2
Data Dari Master : 3
Data Dari Master : 4
Data Dari Master : 5
Data Dari Master : 6
Data Dari Master : 7
Data Dari Master : 8
Data Dari Master : 9
Data Dari Master : 10
Data Dari Master : 1
```

Figure 1.5: Serial Monitor output

The figure above shows the Slave's Serial Monitor after it has successfully received data sent by the Master.

1.5.2 Using Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) uses **Central** and **Peripheral** roles. A **Central/Client** device scans for and connects to a BLE device, while a **Peripheral/Server** performs **advertising** and provides data through the GATT (**Generic Attribute Profile**) structure.

In this example, the **Peripheral/Server** generates a random value and sends it through a notification. The **Central/Client** searches for a device advertising the matching **Service UUID**, connects to it, and receives data through a callback function.

Information
The following website can be used to generate a random UUID:
https://uuidgenerator.net

- **Central/Client**

The Central scans for BLE devices, identifies the Peripheral by its **Service UUID**, establishes a connection, and receives notifications from the characteristic.

BLE Central/Client Libraries, Variables, and Declarations

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress = nullptr;
static BLERemoteCharacteristic *pRemoteCharacteristic = nullptr;
static BLEClient *pClient = nullptr;

static bool doConnect = false;
static bool connected = false;
```

These libraries provide BLE scanning, client-connection, and characteristic-access functions. Pointer variables store the device address, client object, and remote characteristic. The `doConnect` and `connected` variables control the connection process and status.

BLE Central/Client Notification Function

```
static void notifyCallback(
  BLERemoteCharacteristic *pCharacteristic,
  uint8_t *pData,
  size_t length,
  bool isNotify
) {
  String receivedData;

  for (size_t i = 0; i < length; i++) {
    receivedData += static_cast<char>(pData[i]);
  }

  Serial.print("Received from Peripheral: ");
  Serial.println(receivedData);
}
```

The `notifyCallback()` function runs automatically when the Central receives a notification from the Peripheral's characteristic. The `pData` parameter contains the received data as a byte array, while `length` indicates the number of bytes received. Each byte is converted to a character with `static_cast<char>()` and appended to the `receivedData` variable.

After all data has been processed, the resulting value is displayed in the Serial Monitor.

BLE Central/Client Connection Callback

```
class MyClientCallbacks : public BLEClientCallbacks {
  void onConnect(BLEClient *pClient) override {
    connected = true;
    Serial.println("Connected to Peripheral!");
  }

  void onDisconnect(BLEClient *pClient) override {
    connected = false;
    Serial.println("Disconnected from Peripheral!");
  }
};
```

The `MyClientCallbacks` class detects changes in the connection status between the Central and Peripheral. The `onConnect()` function runs when the Central successfully connects to the Peripheral, setting `connected` to `true`. The `onDisconnect()` function runs when the connection is lost, setting `connected` to `false` so that the Central can resume scanning.

BLE Central/Client Scanning Callback

```
class MyAdvertisedDeviceCallbacks
  : public BLEAdvertisedDeviceCallbacks {

  void onResult(BLEAdvertisedDevice advertisedDevice) override {
    Serial.print("Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    if (
      advertisedDevice.haveServiceUUID() &&
      advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ) {
      Serial.println("Target Peripheral found!");
      BLEDevice::getScan()->stop();

      if (pServerAddress != nullptr) {
        delete pServerAddress;
      }

      pServerAddress =
        new BLEAddress(advertisedDevice.getAddress());

      doConnect = true;
    }
  }
};
```

The `MyAdvertisedDeviceCallbacks` class processes each BLE device discovered during scanning. The `onResult()` function runs whenever the scanner finds a BLE device, and the device information is displayed through `advertisedDevice.toString()`. The `haveServiceUUID()` function checks whether the device advertises a Service UUID. The `isAdvertisingService()` function then compares the advertised UUID with the target `SERVICE_UUID`.

When the UUID matches, scanning is stopped with `BLEDevice::getScan()->stop()`. The Peripheral address is stored in `pServerAddress`, and `doConnect` is set to `true` so that the connection process can run in `loop()`.

BLE Central/Client Initialization and Connection

```
bool connectToServer(BLEAddress address) {
  Serial.print("Connecting to Peripheral: ");
  Serial.println(address.toString().c_str());

  pClient = BLEDevice::createClient();
  pClient->setClientCallbacks(new MyClientCallbacks());

  if (!pClient->connect(address)) {
    Serial.println("Connection failed!");
    return false;
  }

  BLERemoteService *pRemoteService =
    pClient->getService(BLEUUID(SERVICE_UUID));

  if (pRemoteService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
```

```
    return false;
  }

  pRemoteCharacteristic =
    pRemoteService->getCharacteristic(BLEUUID(CHARACTERISTIC_UUID));

  if (pRemoteCharacteristic == nullptr) {
    Serial.println("Characteristic not found!");
    pClient->disconnect();
    return false;
  }

  if (!pRemoteCharacteristic->canNotify()) {
    Serial.println("Characteristic does not support notifications!");
    pClient->disconnect();
    return false;
  }

  pRemoteCharacteristic->registerForNotify(notifyCallback);
  connected = true;

  Serial.println("Notification enabled!");
  return true;
}
```

`connectToServer()` connects the Central, locates the service and characteristic, and registers the notification callback. This function is subsequently called from `loop()`.

Central/Client Source-Code Setup

```
void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Central/Client ===");

  BLEDevice::init("ESP32_Central");
  BLEScan *pBLEScan = BLEDevice::getScan();

  pBLEScan->setAdvertisedDeviceCallbacks(
    new MyAdvertisedDeviceCallbacks()
  );
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);

  Serial.println("Scanning for Peripheral...");
  pBLEScan->start(5, false);
}
```

In `setup()`, Serial and BLE are initialized and the previously defined callback is registered with the `pBLEScan` pointer through `setAdvertisedDeviceCallbacks()`. The scan interval, scan window, active scanning, and Peripheral search are then configured.

Loop BLE Central/Client

```
void loop() {
  if (doConnect && pServerAddress != nullptr) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Ready to receive data!");
    } else {
      Serial.println("Connection failed. Scanning again...");
    }

    doConnect = false;
  }

```

```
  if (!connected && !doConnect) {
    BLEDevice::getScan()->start(5, false);
  }

  delay(1000);
}
```

In `loop()`, the Central attempts to connect after the target is found. If it is not connected, scanning starts again.

- **Peripheral/Server**

The Peripheral performs **advertising**, provides a service and characteristic, and sends random values to the Central through notifications.

BLE Peripheral/Server Libraries, Variables, and Declarations

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

BLEServer *pServer = nullptr;
BLECharacteristic *pCharacteristic = nullptr;

bool deviceConnected = false;
bool oldDeviceConnected = false;

unsigned long previousSendTime = 0;
const unsigned long sendInterval = 1000;
```

The BLE Server libraries are used to create the server, service, characteristic, and notification descriptor. Connection variables store the Central's status, while timing variables control the data-transmission interval.

BLE Peripheral/Server Callback

```
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) override {
    deviceConnected = true;
    Serial.println("Central connected!");
  }

  void onDisconnect(BLEServer *pServer) override {
    deviceConnected = false;
    Serial.println("Central disconnected!");
  }
};
```

The server callback runs when the Central connects or disconnects. The `deviceConnected` value is used by `loop()` to determine whether notifications can be sent.

BLE Peripheral/Server Initialization

```
void BLE_init() {
  BLEDevice::init("ESP32_Peripheral");

  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

```

```
  BLEService *pService =
    pServer->createService(BLEUUID(SERVICE_UUID));

  pCharacteristic = pService->createCharacteristic(
    BLEUUID(CHARACTERISTIC_UUID),
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setValue("0");

  pService->start();

  BLEAdvertising *pAdvertising =
    BLEDevice::getAdvertising();

  pAdvertising->addServiceUUID(BLEUUID(SERVICE_UUID));
  pAdvertising->setScanResponse(true);

  BLEDevice::startAdvertising();

  Serial.println("Peripheral ready. Waiting for Central...");
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  BLE_init();
}
```

`BLE_init()` creates the server, service, and characteristic, adds the notification descriptor, and starts advertising. The `setup()` function initializes Serial, seeds the random-number generator for the data-transmission experiment, and configures BLE.

Loop BLE Peripheral/Server

```
void loop() {
  if (
    deviceConnected &&
    millis() - previousSendTime >= sendInterval
  ) {
    previousSendTime = millis();

    String value = String(random(0, 101));

    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();

    Serial.print("Sent: ");
    Serial.println(value);
  }
  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();

    Serial.println("Advertising restarted...");
    oldDeviceConnected = false;
  }
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = true;
  }
}
```

In `loop()`, the Peripheral sends a random value once per second while the Central is connected. If the connection is lost, advertising is restarted so that the device can be discovered and reconnected.

With this flow, the Peripheral sends data and the Central receives it through notifications. The communication does not use a serial stream as Bluetooth Classic does; instead, it uses GATT services, characteristics, and callbacks.

APPENDIX: MODULE 1

Bluetooth Master Source Code

Bluetooth Master

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String masterName = "ESP32-Master"; // ESP Master name
String slaveName = "ESP32-Slave";
bool connected = false;
int value = 0;

void setup() {
  Serial.begin(115200);

  SerialBT.begin(masterName, true); // Start Bluetooth
  Serial.println("The device started in master mode, make sure remote BT device is on!");

  connected = SerialBT.connect(slaveName); // Check the Bluetooth connection
  if(connected) {
    Serial.println("Connected Successfully!");
  } else {
    while(!SerialBT.connected(10 * 1000)) { // Wait for 10 seconds
      Serial.println("Failed to connect!");
      Serial.println("Make sure remote device is available and in range, then restart");
    }
  }
  SerialBT.connect();
}

void loop() {
  if(value <= 10){
    SerialBT.println(value); // Send data through Bluetooth
    Serial.println(value);
    value++;

    delay(1000);
  }
  else {
    value = value - 10;
  }
}
```

Bluetooth Slave Source Code

Bluetooth Slave

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // ESP Slave name
int value = 0;
String slaveValueStr;
int slaveValueInt;

void setup() {
  Serial.begin(115200);
  SerialBT.begin(slaveName); // Start Bluetooth as a receiving device
  Serial.println("The device started, now you can pair it with bluetooth!");
}

void loop() {
  if(SerialBT.available()){
```

```
    slaveValueStr = SerialBT.readStringUntil('\n'); // Read data from the Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data from Master: ");
    Serial.println(slaveValueInt);
  }
}
```

BLE Central/Client Source Code

BLE Central/Client

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress;   // Address of the BLE Peripheral device
static boolean doConnect = false;    // Flag used to start a connection
static boolean connected = false;    // Connection status
static BLERemoteCharacteristic* pRemoteCharacteristic; // Remote characteristic
BLEClient*  pClient;                 // BLE client object

static void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify) {

  // Place control logic here

  String receivedData = "";
  for (size_t i = 0; i < length; i++) {
    receivedData += (char)pData[i];
  }

  Serial.print("Received from Slave: ");
  Serial.println(receivedData);
}

bool connectToServer(BLEAddress pAddress) {
  Serial.print("Connecting to Slave: ");
  Serial.println(pAddress.toString().c_str());

  // Create a new client
  pClient = BLEDevice::createClient();

  // Attempt to connect to the BLE Peripheral address
  if (!pClient->connect(pAddress)) {
    Serial.println("Connection failed!");
    return false;
  }

  Serial.println("Connected to Slave!");

  // Obtain the service from the Peripheral
  BLERemoteService* pRemoteService = pClient->getService(SERVICE_UUID);
  if (pRemoteService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
    return false;
  }

```

```
  // Obtain the characteristic from the service
  pRemoteCharacteristic = pRemoteService->getCharacteristic(CHARACTERISTIC_UUID);
  if (pRemoteCharacteristic == nullptr) {
    Serial.println("Characteristic not found!");
    pClient->disconnect();
    return false;
  }

  // Enable notifications
  if (pRemoteCharacteristic->canNotify())
    pRemoteCharacteristic->registerForNotify(notifyCallback);

  connected = true;
  return true;
}

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
  void onResult(BLEAdvertisedDevice advertisedDevice) {
    Serial.print(" Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    // Check whether the device advertises the required service UUID
    if (advertisedDevice.haveServiceUUID() &&
        advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ){
      Serial.println("Found target device! Stopping scan...");
      BLEDevice::getScan()->stop();

      pServerAddress = new BLEAddress(advertisedDevice.getAddress());
      doConnect = true;
    }
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Master (Central) ===");

  // Initialize the BLE Central
  BLEDevice::init("ESP32_Master");

  // Start scanning
  BLEScan* pBLEScan = BLEDevice::getScan();
  pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);
  pBLEScan->start(5, false);  // Scan for 5 seconds
}

void loop() {
  // When a matching device is found, attempt to connect
  if (doConnect) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Connected & ready to receive data!");
    } else {
      Serial.println("Retrying scan...");
    }
    doConnect = false;
  }

  // If disconnected, scan again
  if (!connected) {
    BLEDevice::getScan()->start(5, false);
  }


```

```
123   delay(1000);
    }
```

BLE Peripheral/Server Source Code

BLE Peripheral/Server

```
1   #include <BLEDevice.h>
2   #include <BLEServer.h>
3   #include <BLEUtils.h>
4   #include <BLE2902.h>
5
6   // Service UUID: identifies the BLE service provided (for example, sensor data)
7   #define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
8
9   // Characteristic UUID: identifies the type of data sent or read within the service
10  #define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"
11
12  BLEServer *pServer = NULL;
13  BLECharacteristic *pCharacteristic = NULL;
14  BLE2902 *pBLE2902;
15
16  bool deviceConnected = false;
17  bool oldDeviceConnected = false;
18
19  class MyServerCallbacks : public BLEServerCallbacks {
20    void onConnect(BLEServer *pServer) {
21      deviceConnected = true;            // Set the status to connected
22      Serial.println("Master connected!");
23    }
24
25    void onDisconnect(BLEServer *pServer) {
26      deviceConnected = false;           // Set the status to disconnected
27      Serial.println("Master disconnected!");
28    }
29  };
30
31  void BLE_init() {
32    // Initialize BLE and assign the device name shown during scanning
33    BLEDevice::init("ESP32_Slave");
34
35    // Create a BLE server (this device acts as the server)
36    pServer = BLEDevice::createServer();
37    pServer->setCallbacks(new MyServerCallbacks()); // Register the callback above
38
39    // Create a BLE service using the UUID
40    BLEService *pService = pServer->createService(SERVICE_UUID);
41
42    // Create a data characteristic within the service
43    // PROPERTY_READ     → The client can read data
44    // PROPERTY_WRITE    → The client can send data to the ESP32
45    // PROPERTY_NOTIFY   → The ESP32 sends data automatically (without an ACK)
46    // PROPERTY_INDICATE → The ESP32 sends data automatically (with an ACK)
47    // PROPERTY_WRITE_NR → The client sends data without confirmation
48    pCharacteristic = pService->createCharacteristic(
49      CHARACTERISTIC_UUID,
50      BLECharacteristic::PROPERTY_NOTIFY
51    );
52
53    // Add a descriptor so the client device can receive notifications
54    pBLE2902 = new BLE2902();
55    pBLE2902->setNotifications(true);
56    pCharacteristic->addDescriptor(pBLE2902);
57
58    // Start the BLE service
59    pService->start();
60
```

```
  // Configure advertising so that other devices can discover this device
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  BLEDevice::startAdvertising();

  Serial.println("Ready to connect!");
}

void setup() {
  Serial.begin(115200);
  BLE_init();
}

void loop() {
  // Data-transmission logic can be placed here
  String value = String(random(0,101));

  if (deviceConnected) {
      // This API sends the value as a null-terminated C string (\0)
      // Convert the Arduino String object to a C string before transmission
      pCharacteristic->setValue(value.c_str());
      pCharacteristic->notify();
      Serial.print("Sent: ");
      Serial.println(value.c_str());
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();
    Serial.println("Advertising after disconnect...");
    oldDeviceConnected = deviceConnected; // Update the previous connection status
  }

  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
  }

  delay(100);
}
```